

CS 3510 Algorithms: Quicksort, Quickselect, and Max Subarray Sum

Aaron Hillegass
Georgia Tech

The one that stumped me

$$O(n^a) < (\log n)^{\log n} = n^{\log \log n} < O(b^n) \text{ for any } a \text{ or } b$$

How do we know they are equal? $a^{\log b} = b^{\log a}$

$O(n^a) < n^{\log \log n}$ because a is a constant and $\log \log n$ grows.

To show $n^{\log \log n} \leq b^n$, take log of both sides: $(\log \log n)(\log n) \leq n \log b$

Ignore the constant $\log b$:

$$\lim_{n \rightarrow \infty} \frac{(\log \log n)(\log n)}{n} < \lim_{n \rightarrow \infty} \frac{(\log n)(\log n)}{n} = \lim_{n \rightarrow \infty} \frac{(2 \log n)}{n}$$

Apply l'Hopital's rule:

$$\lim_{n \rightarrow \infty} \frac{2}{n} = 0$$

So b^n grows faster than $n^{\log \log n}$.

Invented in 1959 by Tony Hoare

Divide-and-Conquer

Usually a little quicker than Mergesort or Heapsort

Sorts in-place – no additional memory requirements

Quicksort Algorithm

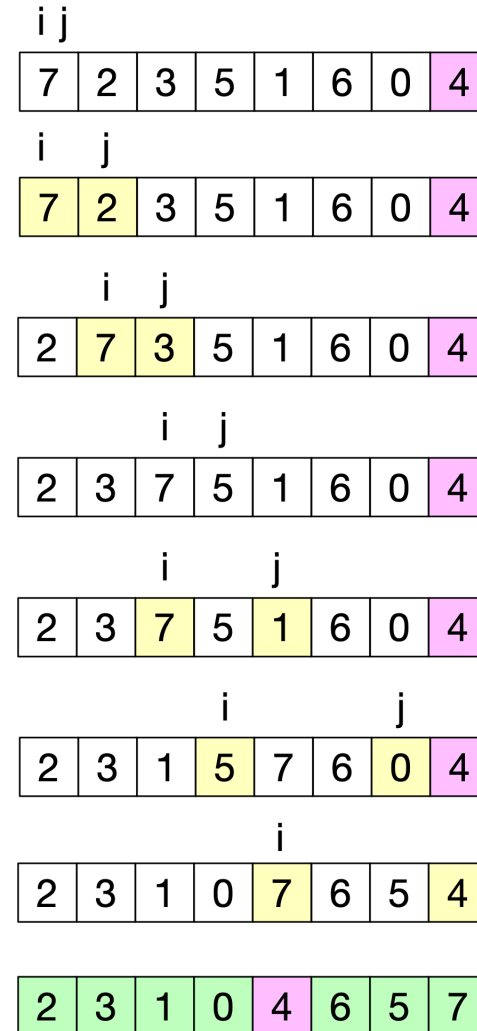
- Partition
 - Pick a “pivot” from the data.
 - Move through the array swapping as necessary to move things less than the pivot to one end and greater than the pivot to the other. Note: $O(n)$
 - Swap the pivot into the border between “Smaller Zone” and “Bigger Zone”
 - Return the new location of the pivot
- Continue
 - Partition everything to the left of the pivot
 - Partition everything to the right of the pivot

Partition

```

1 define partition( $A$ , lo, hi):
2     pivot :=  $A$ [hi]
3      $i$  := lo
4     for  $j$  in  $i$  to hi - 1:
5         if  $A[j] \leq$  pivot:
6             swap  $A[i]$  with  $A[j]$ 
7              $i$  :=  $i$  + 1
8     swap  $A[i]$  with  $A$ [hi]
9     return  $i$ 

```

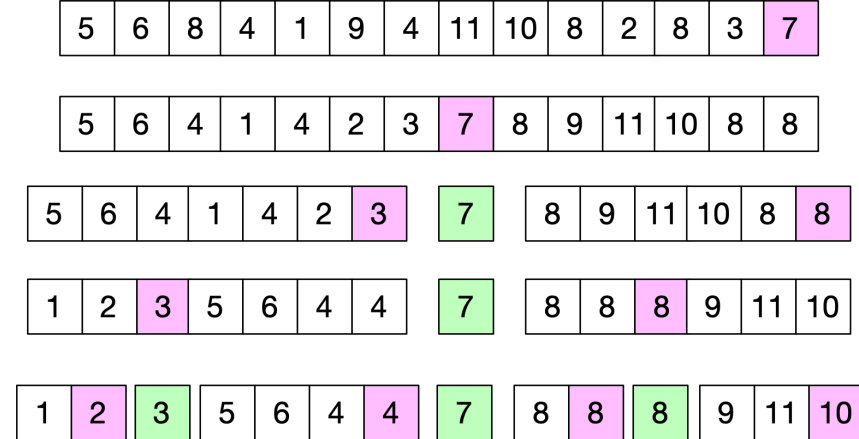


Quicksort

```

1 define qsort( $A$ , lo, hi):
2   if hi  $\leq$  lo:
3     return
4    $p := \text{partition}(A, \text{lo}, \text{hi})$ 
5   qsort( $A$ , lo,  $p - 1$ )
6   qsort( $A$ ,  $p + 1$ , hi)

```



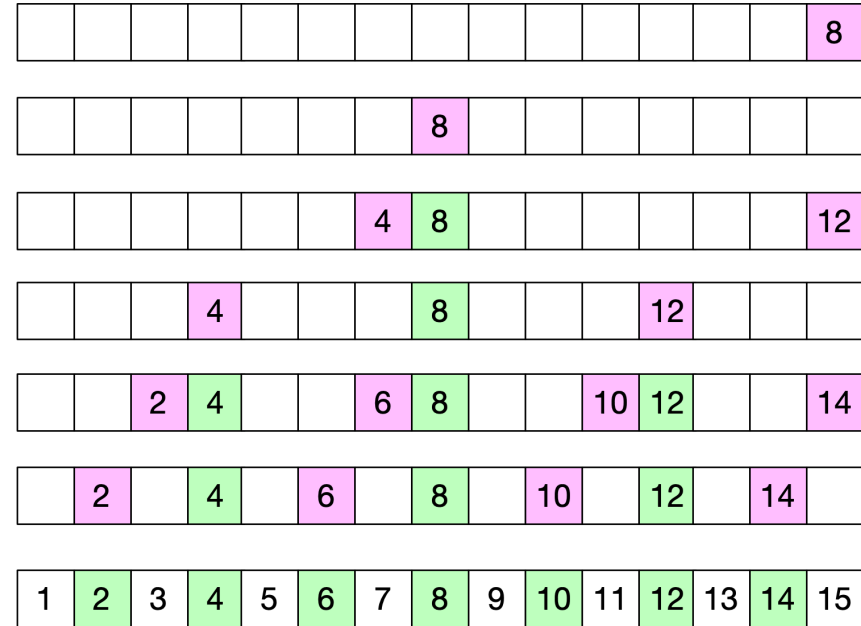
Best Case Analysis

7/23

You pick the median every time!

$$T(n) = 2T\left(\frac{n-1}{2}\right) + O(n) < 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) \in O(n \log n)$$



Worst Case Analysis

You pick the min/max every time!

$$T(n) = T(n - 1) + O(n)$$

$$T(n) \in O(n^2)$$



...

Average Case

$$T(n) = \frac{1}{n} \sum_{i=0}^{n-1} [T(i) + T(n-i)] + O(n) = \frac{2}{n} \sum_{i=0}^{n-1} T(i) + n$$

$$nT(n) = 2 \sum_{i=0}^{n-1} T(i) + n^2$$

$$(n-1)T(n-1) = 2 \sum_{i=0}^{n-2} T(i) + (n-1)^2$$

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 1$$

$$nT(n) = (n+1)T(n-1) + 2n$$

Divide by $n(n+1)$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2}{n+1} \text{ evaluated at } n-1: \frac{T(n-1)}{n} = \frac{T(n-2)}{n-1} + \frac{2}{n}$$

$$\frac{T(n)}{n+1} = \frac{T(n-2)}{n-1} + \frac{2}{n} + \frac{2}{n+1}$$

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2 \sum_{i=3}^{n+1} \frac{1}{i}$$

$$H_n = \sum_{i=1}^n \frac{1}{i}$$

$$H_n \in O(\log n)$$

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2 \sum_{i=3}^{n+1} \frac{1}{i}$$

$$\frac{T(n)}{n} = \frac{T(1)}{2} + 2O(\log n)$$

$$T(n) = O(n \log n)$$

$$\text{Quicksort} \in O(n \log n)$$

How could you find the k -th smallest item in an array?

```
1 define qselect( $A$ , lo, hi,  $k$ ):  
2   if hi == lo:  
3     | return  $A$ [lo]  
4    $p :=$  partition( $A$ , lo, hi)  
5   if  $k == p$ : return  $A$ [ $p$ ]  
6   if  $k < p$ :  
7     | return qselect( $A$ , lo,  $p - 1$ ,  $k$ )  
8   if  $k > p$ :  
9     | return qselect( $A$ ,  $p + 1$ , high,  $k$ )
```

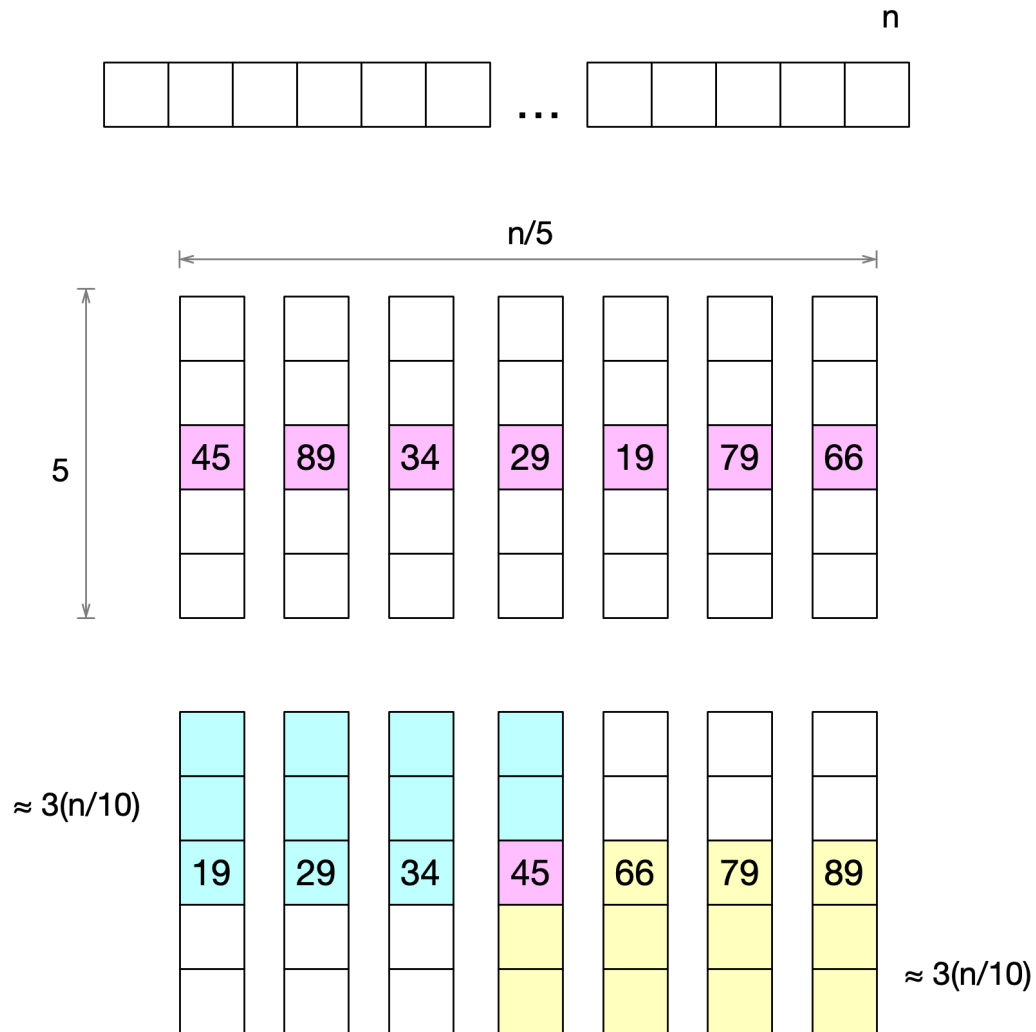
worst pivots: $O(n^2)$

best pivots: $O(n)$

Picking a good pivot is a *big* deal.
Can we eliminate the worst cases?

Median of Medians

14/23



Guarantees that pivot is between 30th and 70th percentile.

We will prove three things. For all A :

- Median of Medians runs in $O(n)$ if we use Quickselect to find the medians.
- Quickselect runs in $O(n)$ if use Median of Medians to find pivots.
- Quicksort runs in $O(n \log n)$ if we use Median of Medians to find pivots.

Let the time to run Quickselect be given by $T(n)$.

Doing a partition is $O(n)$

The time for MoM to find the median of the medians using Quickselect would be $T(\frac{n}{5})$

We know that the worst case using MoM is that the scan only cuts the search space by 30%.

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

Can we find a function T that satisfies this? Let's try a linear function: $T(n) = cn$

$$cn \leq an + c\left(\frac{n}{5}\right) + c\left(\left(\frac{7}{10}\right)n\right) = \left(1 + c\left(\frac{9}{10}\right)\right)n \text{ for some } a > 0$$

$$c \leq a + \frac{9}{10}c$$

$$c \leq 10a$$

$$T(n) \leq 10an$$

$T(n)$ is linear. That is, Quickselect works in $O(n)$.

Interlocking Proofs

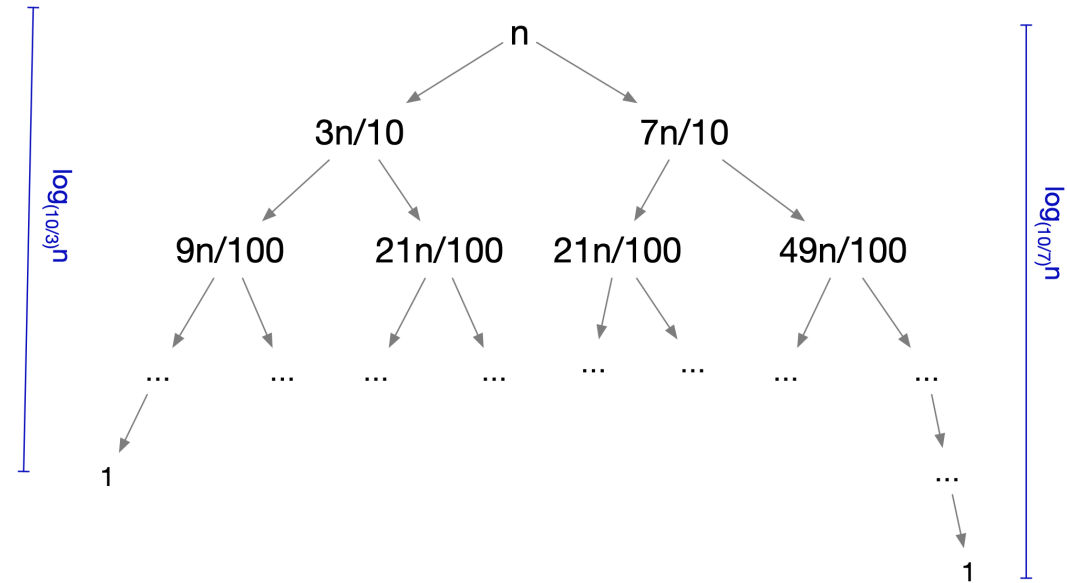
Quickselect find medians $O(n)$.

Median of Medians for an array of n ,
requires $(\frac{n}{5} \text{ medians of } 5) + (1 \text{ median of } \frac{n}{5})$.
It is $O(n)$.

Worst case Quicksort using MoM:

$$T(n) = T\left(\frac{7n}{10}\right) + T\left(\frac{3n}{10}\right) + O(n)$$

Quicksort using Median of Medians is $O(n \log n)$



First or last: Terrible on lists that are already sorted

Random: Usually $O(n \log n)$

Median of Three: Almost always $O(n \log n)$

Median of Medians: Guaranteed $O(n \log n)$

In practice: Median of Three

The Maximum Subarray Sum Problem

Given an array $A[1...n]$ of numbers, find a subarray with the max sum. Or find $1 \leq i \leq j \leq n$ that maximizes the sum:

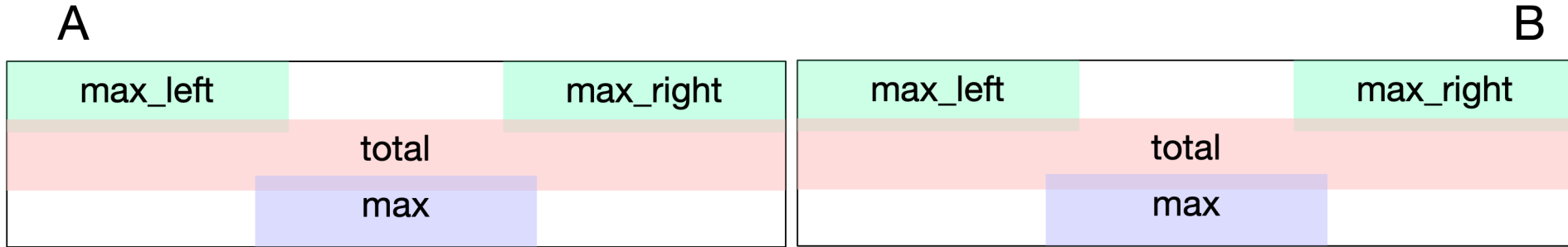
$$\sum_{k=i}^j A[k]$$

Brute force? $O(n^6)$

Grenander? $O(n^2)$

Shamos Divide-and-Conquer? $O(n \log n)$

Divide and Conquer for Max Subarray



Joining two?

- $\text{max_left} := \max(A.\text{max_left}, A.\text{total} + B.\text{max_left})$
- $\text{max_right} := \max(B.\text{max_right}, B.\text{total} + A.\text{max_right})$
- $\text{total} := A.\text{total} + B.\text{total}$
- $\text{max} := \max(\text{max_left}, \text{max_right}, A.\text{max}, B.\text{max}, A.\text{max_right} + B.\text{max_left})$

Array of one?

- All of the above are equal: value in the array.

Divide and Conquer for Max Subarray

```
1 define analyze( $A$ ):
2    $n := \text{len}(A)$ 
3   if  $n == 1$ :
4     | return ( $ml : A[1], mr : A[1], \text{total} : A[1], mx : A[1]$ )
5    $c := \lfloor \frac{n}{2} \rfloor$ 
6    $L := A[1..c], R := A[(c + 1)..n]$ 
7   ( $lml, lmr, lttotal, lmx := \text{analyze}(L)$ )
8   ( $rml, rmr, rttotal, rmx := \text{analyze}(R)$ )
9   return ( $ml : \max(lml, lttotal + rml),$ 
10    |  $mr : \max(rmr, rttotal + lmr),$ 
11    |  $\text{total} : lttotal + rttotal,$ 
12    |  $mx : \max(lml, rml, lmx, rmx, lrm + rlm))$ 
```

```
1 define max_subsum( $A$ ):
2   | ( $_, _, _, mx$ ) := analyze( $A$ )
3   return  $mx$ 
```

Divide and Conquer for Max Subarray

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

$$O(n)$$

Questions?

Slides by Aaron Hillegass